Software Architecture and Design Specification (SAD)

Smart Traffic Complaint and Monitoring System

Project team: Arya, Ansh, Arnav, Avyukt

Technology baseline: MERN (MongoDB, Express.js, React, Node.js), with Python FastAPI and PyTorch for image-based road classification.

Document status: Updated to describe the delivered implementation known as of 30 October 2026. Items not independently verified are identified as design intent or require confirmation.

1. Introduction

This document describes the architecture and design of the Smart Traffic Complaint and Monitoring System. The original MERN design has been extended with optional complaint-image upload and an AI inference service for road-image classification. The architecture supports citizen complaint submission and tracking, administrator complaint management, and AI-assisted classification. AI output is advisory and does not replace administrator review.

2. Architecture Overview

2.1 Architecture pattern

The application uses a client-server, three-tier architecture for the core MERN application: React presentation layer, Node.js/Express application/API layer, and MongoDB persistence layer. An additional Python FastAPI inference service is called by the Express API when a complaint includes an image. In local development, the frontend uses port 5173, the Express API uses port 5050, and the FastAPI service uses port 8000. Deployment transport and TLS configuration must be set for the target hosting environment.

2.2 Components

• React citizen UI: registration/login, complaint submission, complaint list and status view; supports selecting an optional JPG/PNG image and previewing it before submission.
• React administrator UI: complaint list, filters, priority/status management and dashboard; displays an uploaded image and available AI prediction details.
• Express REST API: routes, request validation, authentication, authorization, complaint persistence and response formatting.
• Authentication/authorization middleware: validates identity and enforces citizen/admin permissions on protected routes.
• Complaint handling: creates and retrieves complaint records, validates category and required fields, accepts image uploads through Multer, and calls the inference service when an image is supplied.
• MongoDB: stores user accounts and complaint documents, including image URL and AI prediction fields.
• Upload storage: image files are stored under the server uploads directory and exposed through the `/uploads` static route.
• FastAPI inference API: exposes health and prediction endpoints; accepts an image at `POST /predict` and returns the predicted category, confidence and class probabilities.
• ResNet18 classifier: PyTorch model loaded from `dl/models/best_model.pth`; class outputs are mapped to Road Damage, Normal Road and Pothole.

2.3 Architecture flow

1. A citizen submits complaint fields and optionally attaches a JPG/PNG image.
2. The React client sends the complaint as multipart form data to the Express complaints endpoint.
3. Express validates the request and stores the complaint and uploaded-image reference.
4. If an image is present, Express forwards it to FastAPI `/predict`.
5. FastAPI preprocesses the image and runs the ResNet18 model.
6. Express stores the returned category, confidence and probability values in the complaint's AI prediction fields.
7. The API returns the saved complaint. If AI inference fails or times out, the complaint is retained and the AI prediction status is recorded as Failed.

2.4 Data model

Entity	Fields / design	Notes
User	_id, name, email, passwordHash, role, createdAt	Email unique; role distinguishes citizen and admin. Passwords are hashed; plaintext passwords are not stored.
Complaint	_id, citizenId, category, description, locationDescription, status, priority, assignedTo, createdAt, updatedAt, imageUrl, aiPrediction	citizenId links to submitting user. status and priority use allowed values. imageUrl is optional. aiPrediction stores status, category, confidence and probabilities.

2.5 AI prediction data

The complaint's `aiPrediction` object records an inference status (`Not Requested`, `Completed`, or `Failed`), predicted category, confidence value, and class probabilities. These values are optional and are populated only when image inference is requested and a result is returned. The classifier's confidence is a model score, not a guarantee of correctness.

2.6 Architecture security

• Hash passwords using the configured password-hashing library; never store plaintext passwords.
• Validate authentication and authorization on the server for protected routes.
• Enforce complaint ownership when citizens request complaint details.
• Restrict administrator operations to administrator accounts.
• Validate required fields, allowed complaint categories, status/priority values and uploaded file type/size.
• The image upload path accepts JPG/PNG files and applies a 10 MB limit in the complaint upload route.
• Use environment variables for secrets and service configuration; do not commit secrets or local `.env` files.
• Return controlled error responses and keep diagnostic detail in protected server logs.
• Treat uploaded images and AI output as untrusted input/data; do not use a model prediction as an authorization or workflow decision.

2.7 Requirement traceability

Component / design element	Requirements supported
React citizen UI	FR-01–07; NFR-02, NFR-04
React administrator UI	FR-08–13
Express API and complaint handling	FR-03–14; NFR-01, NFR-06, NFR-07, NFR-10
Authentication/authorization middleware	FR-01, FR-02, FR-06, FR-15; SR-01, SR-02; NFR-08, NFR-09
MongoDB data model	FR-04–13; NFR-07; SR-03, SR-05
Image upload and AI inference extension	Complaint image submission and AI-assisted road category prediction; detailed acceptance tests are tracked in the Software Test Plan.
Validation and safe error handling	FR-14; NFR-06, NFR-10; SR-04–SR-06

3. Detailed Design

3.1 Sequence: citizen submits complaint with optional image

Citizen → React UI: enter category, description and location; optionally select an image.
React UI → Express API: submit multipart form data to the complaints endpoint.
Express API: authenticate user, validate fields/category and validate uploaded file.
Express API → MongoDB: save complaint and image URL.
Express API → FastAPI: when an image exists, send it to `/predict`.
FastAPI → Express API: return category, confidence and probabilities.
Express API → MongoDB: save AI prediction status and result.
Express API → React UI: return created complaint and any available prediction details.
Failure path: if inference fails, the complaint remains saved and AI status is set to Failed.

3.2 Sequence: administrator updates complaint status

Administrator → React Admin UI: select complaint and choose a new status or priority.
React Admin UI → Express API: send the authenticated update request to the admin route.
Express API: verify administrator permission and validate update values.
Express API → MongoDB: update the complaint document.
MongoDB → Express API: return the updated record.
Express API → React Admin UI: return updated complaint data for display.

3.3 API design

Method / endpoint	Access	Purpose
POST /api/auth/register	Public	Register a user; return safe user fields.
POST /api/auth/login	Public	Authenticate and return the configured token/session response.
POST /api/auth/logout	Authenticated / if implemented	Logout behavior depends on the configured token/session implementation.
POST /api/complaints	Citizen	Create complaint; accepts multipart form fields and optional `image` file.
GET /api/complaints	Authenticated	Retrieve complaint list according to the route's access/ownership logic.
GET /api/complaints/:id	Authenticated / owner or admin	Retrieve a complaint subject to ownership/role checks, where implemented.
GET /api/admin/complaints	Admin	List and filter complaints.
PATCH /api/admin/complaints/:id	Admin	Update allowed complaint status, priority or assignment fields as supported by the route.
GET /api/admin/dashboard	Admin	Return aggregate complaint counts.
GET /uploads/:filename	Public static file route	Serve an uploaded complaint image by its stored image URL.
GET /health	Service health	FastAPI health check; reports model-loaded state and inference device.
POST /predict	Internal service call	FastAPI accepts multipart image field `file`; returns prediction category, confidence and probabilities.

Implementation note: the API table reflects the known route structure and service integration. Confirm exact route names and authorization behavior against the current route files before treating every row as a final API contract.

3.4 Example complaint request and response

Multipart request fields: `category`, `description`, `location`, and optional file field `image` (JPG/PNG, up to 10 MB).
Illustrative AI result: category `Pothole`, confidence `0.98`, with class probabilities for Road Damage, Normal Road and Pothole. Values shown are examples only; actual prediction depends on the submitted image.

3.5 Error handling

HTTP status	Use
400 Bad Request	Missing required fields, invalid complaint category, invalid status/priority, malformed request or rejected upload.
401 Unauthorized	Missing or invalid authentication.
403 Forbidden	Authenticated user lacks required role or permission.
404 Not Found	Complaint identifier or requested resource does not exist or is not available to requester.
409 Conflict	Duplicate email or conflicting resource state, where applicable.

HTTP status	Use
500 Internal Server Error	Unexpected failure; response must not reveal stack traces or secrets.

4. Deployment and Maintenance

• Development uses separate frontend, Express API and FastAPI service processes, with MongoDB configured separately.
• Configure database connection strings, signing secrets and `DL_API_URL` through environment variables. Do not commit real secrets.
• The inference URL defaults to `http://127.0.0.1:8000/predict` in the backend when not overridden.
• The FastAPI process requires Python dependencies and the trained model weights. Local virtual environments and datasets are not source-controlled runtime deliverables.
• Uploaded images are stored in `server/uploads`; this directory is excluded from Git and must be provisioned/persisted appropriately in deployment.
• Review changes through pull requests and run automated checks before merging.
• GitHub Actions workflow `TrafficCare CI` runs backend Jest tests and the frontend production build on pushes and pull requests targeting `main`. The workflow does not currently run the FastAPI model or deploy the application.

5. Implementation Deviations and Current Status

Baseline design	Delivered implementation / status
Three-tier MERN application only.	Extended with a separate Python FastAPI inference service and PyTorch ResNet18 classifier.
Complaint fields only; no image field specified.	Optional complaint image upload implemented using Multer; image URL stored in complaint record.
Complaint record has no AI result.	Complaint model extended with AI prediction status, category, confidence and probabilities.
Complaint form submits ordinary request data.	Complaint form sends multipart form data when submitting an optional image.
Admin UI displays complaint details and status.	Admin UI also displays uploaded image and AI prediction information.
Automated checks to be run before merge.	TrafficCare CI workflow added; latest observed run passed backend tests and frontend build on 30 October 2026.

6. Design Constraints and Items Requiring Verification

• Confirm exact endpoint paths, logout semantics and all role/ownership checks against the current source code before final submission.
• Verify image persistence and static image access in the intended deployment environment; local filesystem storage may require persistent volume or object storage in production.
• Execute end-to-end tests for successful AI inference and inference failure handling; a passing CI workflow alone does not verify the FastAPI service.
• Evaluate model quality separately on an appropriate held-out dataset. Confidence scores must not be interpreted as guaranteed correctness.
• Confirm TLS/HTTPS, production CORS policy, secret management, backup and monitoring configuration when deployment infrastructure is selected.

7. Review and Approval

This document describes the implementation state known from the project updates through 30 October 2026. Team review and approval signatures should be added according to the institution's submission process. Any later architectural change should be recorded in the deviation section and reflected in the UML diagrams and Software Test Plan.